Rectificar o ajustar: detección de objetos en cámaras *fisheye* de vehículos

Franco Marcelo Morero, Nicolás Granato, Marcos Levi Riveros Koloszwa y Tadeo Riveros Koloszwa
Carrera de Especialización en Inteligencia Artificial (CEIA), Facultad de Ingeniería, Universidad de Buenos Aires
Visión por Computadora II, grupo 1

***Resumen*—Las cámaras de visión envolvente de los vehículos usan lentes *fisheye* de casi 190°, que deforman los objetos hacia los bordes de la imagen. Un detector preentrenado en imágenes comunes pierde allí la mayor parte de su desempeño. Se compararon dos formas de adaptarlo: rectificar la imagen con una proyección cilíndrica y hacer *fine-tuning* sobre las imágenes *fisheye*. Las cuatro combinaciones de las dos estrategias usan el mismo YOLO11m, los mismos datos de WoodScape y la misma métrica, el mAP por anillos del ángulo de incidencia, sobre 1000 imágenes de prueba. Sin adaptar, el detector pierde el 69 % de su mAP@50 entre el centro y la periferia. El *fine-tuning* sube la periferia de 0,170 a 0,603, reduce esa caída al 14 % y no agrega latencia. Rectificar mejora al detector preentrenado, pero empeora al ajustado, porque las cajas que vuelven a la imagen original quedan más grandes que los objetos. Un aumento de datos con distorsión radial no cambia el resultado con las mismas lentes, pero reduce a la mitad la pérdida con lentes distintas.**

***Palabras clave*—cámaras *fisheye*, detección de objetos, *fine-tuning*, proyección cilíndrica, WoodScape, YOLO11**

I. Introducción

Los sistemas de visión envolvente (*surround view*) combinan cuatro cámaras *fisheye*, una frontal, una trasera y una en cada espejo, que cubren 360° alrededor del vehículo. Con ellas funcionan el estacionamiento automático, la asistencia en atascos y la conducción urbana [2]. La lente sigue un modelo de proyección polinomial en θ, como el de Kannala y Brandt [7]. En WoodScape es de cuarto orden, $\rho(\theta) = k_1\theta + k_2\theta^2 + k_3\theta^3 + k_4\theta^4$ [1], y en él la deformación de un objeto depende del ángulo de incidencia θ respecto del eje óptico. Un peatón a 20° y el mismo peatón a 85° tienen geometrías muy distintas en la imagen. Los detectores disponibles se preentrenan en imágenes de perspectiva, como las de COCO (*Common Objects in Context*) [8], y por eso resuelven peor la periferia, que es la zona más importante para la seguridad.

La industria automotriz ya trabaja sobre este problema. Valeo publicó WoodScape [1], el primer conjunto de datos público extenso de cámaras *fisheye* automotrices. Sobre ese conjunto, su equipo desarrolló OmniDet (2021) [3], una red de percepción multitarea para las cuatro cámaras que funciona en tiempo real en *hardware* embebido automotriz (NVIDIA Jetson AGX con TensorRT). Fuera del vehículo, FishEye8K (2023) [6] reúne 8000 imágenes de 18 cámaras *fisheye* que monitorean el tránsito en calles e intersecciones de Hsinchu, Taiwán, para detectar autos, autobuses, camiones, bicicletas y motos, y peatones. En los dos casos el detector trabaja sobre la imagen deformada.

Las soluciones se agrupan en dos familias. La primera rectifica la imagen y usa el detector sin cambios. Plaut et al. [5] mostraron que una proyección cilíndrica permite aplicar detectores entrenados solo con imágenes de perspectiva. La segunda adapta el modelo con datos *fisheye*, y Rashed et al. [4] señalaron además que la caja rectangular representa mal a los objetos deformados. Para un equipo que parte de un detector preentrenado, la pregunta práctica es cuál de las dos conviene y si vale la pena combinarlas. Los trabajos previos suelen evaluar una sola estrategia, con modelos, datos o métricas distintos, y eso impide compararlas.

El objetivo del trabajo es medir cuánto aporta rectificar, cuánto aporta el *fine-tuning* y cómo interactúan, con el mismo detector, los mismos datos y la misma métrica, y con foco en la periferia y en la latencia. Las contribuciones son las siguientes:

- Un diseño factorial 2×2 que cruza rectificar o no con hacer *fine-tuning* o no, para separar los dos efectos y estimar su interacción.
- Una métrica que separa los objetos según su ángulo de incidencia θ, es decir, según qué tan lejos del centro de la lente caen. Sigue el esquema de evaluación de COCO, está validada contra `pycocotools` y usa una partición controlada para que no haya imágenes casi iguales entre entrenamiento y prueba.
- La evidencia de que rectificar resta una vez que el modelo se adapta, con una explicación cuantitativa: el límite de IoU (*intersection over union*) que impone el regreso de las cajas a la imagen original.
- Un aumento de datos con distorsión radial que no cambia el desempeño con las lentes del conjunto de datos, pero reduce a la mitad la pérdida con otras lentes.

II. Metodología, diseño y desarrollo

II-A. Datos y partición

Se usaron las 8234 imágenes públicas de WoodScape [1], de 1280×966 píxeles, tomadas por las cámaras FV (*front view*, frontal), MVL (*mirror view left*, espejo izquierdo), MVR (*mirror view right*, espejo derecho) y RV (*rear view*, trasera), con 23 juegos de calibración intrínseca. Las 43 etiquetas de instancia se agrupan en tres clases: `vehicle` (`car`, `van`, `bus`, `truck`, `trailer` y `caravan`), `person` (`person` y `rider`) y `two_wheeler` (`bicycle` y `motorcycle`). La caja de referencia es la envolvente del polígono de cada objeto.

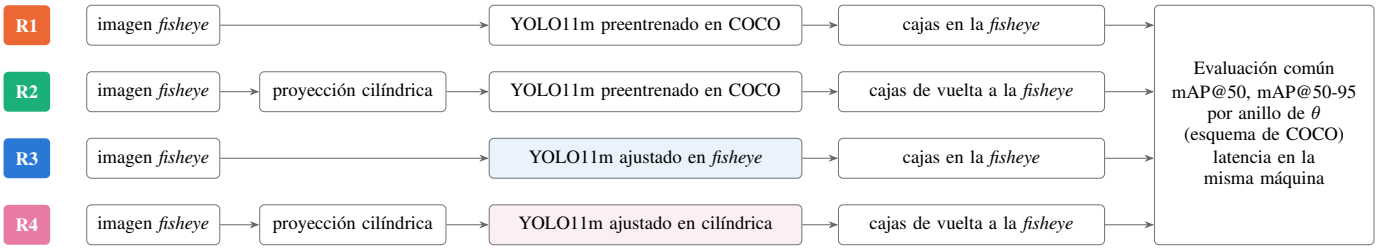


Figura 1. Diseño factorial 2×2. R1 aplica el detector preentrenado sobre la imagen original. R2 aplica el mismo detector sobre la proyección cilíndrica y devuelve las cajas a la imagen *fisheye*. R3 hace *fine-tuning* sobre la imagen *fisheye* y R4, sobre la proyección cilíndrica. Las cuatro ramas se evalúan con la misma métrica sobre la imagen original.

Se descartan las cajas menores a 300 px^2 o con un lado menor a 8 px.

WoodScape proviene de grabaciones continuas, así que un muestreo aleatorio podría repartir cuadros casi idénticos entre entrenamiento y prueba. Con *hashes* perceptuales de 256 bits se verificó que la distancia mediana entre cada imagen de prueba y su vecina más cercana de entrenamiento es de 39 bits (15,2 %). El único par sospechoso quedó en cuarentena. La partición final, estratificada por cámara, tiene 6148 imágenes de entrenamiento, 1085 de validación y 1000 de prueba con 11 910 objetos. Las imágenes se verifican por sha256 y nunca se recomprimen. La prueba no se usó para elegir *checkpoint* ni hiperparámetros. Cada rama se evaluó sobre ella con sus pesos finales.

II-B. Métrica estratificada por ángulo de incidencia

Para cada caja se calcula el ángulo θ de su centro a partir de la calibración de su imagen. Se usa θ y no la distancia en píxeles al centro porque es lo que determina la deformación y vale igual para cámaras con calibraciones distintas. Las cajas se reparten en tres anillos: centro, medio y periferia. Los cortes, 68,3° y 80,7°, dejan 3970 objetos de la prueba en cada anillo y son los mismos para todas las ramas.

Cada anillo se evalúa como COCO evalúa cada rango de tamaño (`areaRng`), con θ en lugar del área. Los objetos de otros anillos se marcan *ignore*, así que detectarlos no cuenta como acierto ni como falso positivo. Como control se usa también un esquema de filtrado, que descarta las cajas de referencia y las detecciones cuyo centro cae fuera del anillo.

Se reporta el mAP (*mean average precision*) con umbral de IoU de 0,5 (mAP@50) y promediado entre 0,5 y 0,95 (mAP@50-95), con la configuración de COCO y un umbral de confianza de 0,01. El evaluador da los mismos valores que `pycocotools`. La latencia se mide por imagen en la misma máquina para las cuatro ramas, una Apple M4 Max con MPS (*Metal Performance Shaders*), después de diez pasadas de calentamiento y sin contar la lectura del disco.

II-C. Las cuatro ramas

Las cuatro ramas (Fig. 1) usan YOLO11m [9], con 20,1 M parámetros, 68,1 GFLOPs y entrada de 640 px. Se probaron ocho detectores preentrenados: YOLO11 en sus cinco tamaños, dos Faster R-CNN y una Mask R-CNN. YOLO11m fue el de mejor equilibrio entre precisión y latencia. Además, es un detector de una etapa, apto para una ECU (*electronic control unit*). Todas las ramas se evalúan sobre la imagen *fisheye* original, que es la que usa el vehículo.

R1, sin adaptar. El detector se aplica directo sobre la imagen. Sus clases de COCO se traducen a las del trabajo: `car`, `bus` y `truck` pasan a `vehicle`, y `bicycle` y `motorcycle` pasan a `two_wheeler`. La clase `person` se mantiene.

R2, rectificar. La imagen pasa a una proyección cilíndrica de 190° horizontales [5], recortada a una banda de $\pm 45°$ alrededor del horizonte. Esa banda contiene el centro del 95,7 % de las cajas de prueba. No se usó la proyección de perspectiva porque lleva los píxeles cercanos a 90° a distancias infinitas [5] y obliga a recortar la periferia, que es la zona en estudio.[1] Para volver a la imagen *fisheye*, cada caja detectada se recorre con 20 puntos por lado. Se descartan los puntos que caen fuera de la imagen y se toma la caja que encierra al resto.

R3, adaptar. Se hace *fine-tuning* de YOLO11m sobre las imágenes *fisheye*, desde los pesos de COCO y con una cabeza nueva para las tres clases.

R4, ambas. Se hace *fine-tuning* sobre la proyección cilíndrica de R2. Para obtener las etiquetas se proyecta el polígono de cada objeto y se toma su caja en la imagen cilíndrica. Proyectar la caja de referencia la agrandaría en la periferia.

R3 y R4 se entrenan igual. Usan 30 épocas, un lote de 64 imágenes por acumulación de gradiente, AdamW con tasa de aprendizaje de $1{,}43 \times 10^{-3}$ y los aumentos de datos por defecto de Ultralytics: mosaico, cambios de color, espejado, escala y traslación, sin mosaico en las 10 últimas épocas. En cada rama se guarda la época con mejor mAP@50-95 en validación. R3 se entrenó en una Apple M4 Max (3,4 h), con 16 imágenes por paso. R4 se entrenó en una GTX 1660 de 6 GB, con 2 imágenes por paso por falta de memoria y sin precisión mixta. También cambia la semilla.

II-D. Extensión: aumento de datos con distorsión radial

El modelo de R3 aprende las lentes de WoodScape. Para que funcione también con otras lentes, se entrenó una variante que

[1] E. Plaut explica con ejemplos por qué la proyección de perspectiva no puede representar ángulos mayores a 90° y la cilíndrica sí, en su tutorial sobre visión por computadora con cámaras *fisheye*: https://plaut.github.io/fisheye_tutorial/.

Tabla I
RESULTADOS EN LAS 1000 IMÁGENES DE PRUEBA (MAP POR ANILLO DE θ, ESQUEMA DE COCO). LA CAÍDA ES LA DEL MAP@50 DEL CENTRO A LA PERIFERIA. LA LATENCIA ES LA MEDIA POR IMAGEN EN LA MISMA MÁQUINA.

	Global		mAP@50 por anillo				
Rama	mAP@50	50-95	centro	medio	perif.	caída	ms
R1 sin adaptar	0,378	0,239	0,541	0,432	0,170	69 %	25,0
R2 cilíndrica	0,418	0,155	0,528	0,420	0,316	40 %	26,6
R3 *fine-tuning*	**0,659**	**0,430**	**0,701**	**0,670**	**0,603**	**14 %**	**22,9**
R4 ambas	0,522	0,172	0,642	0,522	0,412	36 %	28,6

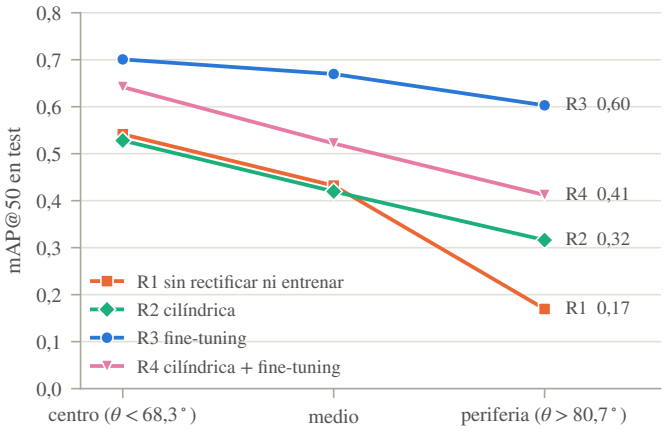


Figura 2. mAP@50 por anillo de ángulo de incidencia en la prueba. R3, el *fine-tuning* sin rectificar, es la única rama que se mantiene casi plana hacia la periferia.

deforma algunas imágenes al azar durante el entrenamiento. La deformación es una distorsión radial simétrica con un solo coeficiente k, del tipo que Brown describe con los coeficientes $K_1, K_2, \ldots$ [10]. Cada píxel a distancia r del centro óptico toma el valor del punto a distancia

$$r_{\text{src}} = r\,\frac{1 + kr^2}{1 + k} \tag{1}$$

donde r vale 1 en la esquina más lejana. Con $k > 0$ la imagen se ve más *fisheye* y con $k < 0$, menos. En cada imagen se aplica con probabilidad 0,5 y con k elegido al azar entre $-0{,}2$ y 0,3, rango en el que la transformación no se pliega sobre sí misma. Las cajas se deforman igual que la imagen. Esta variante se entrenó con la misma configuración que R3.

III. RESULTADOS EXPERIMENTALES Y ANÁLISIS

III-A. Resultados por anillo

La Tabla I y la Fig. 2 muestran los resultados de las cuatro ramas. La caída es cuánto baja el mAP@50 del centro a la periferia, en porcentaje. Sin adaptar, R1 cae un 69 %. El problema no es de este detector en particular. YOLO26m [11], una generación posterior, obtiene en la periferia casi lo mismo que YOLO11m, 0,171 contra 0,160 con el esquema de filtrado.

La caída se debe a la deformación y no al tamaño de los objetos. La Tabla II separa los objetos por tamaño, con los rangos de área de COCO. En R1, los grandes bajan de 0,841

Tabla II
MAP@50 POR ANILLO Y TAMAÑO DE OBJETO (RANGOS DE ÁREA DE COCO: PEQUEÑO $< 32^2$, MEDIANO $< 96^2$ PX). ENTRE PARÉNTESIS, LA CANTIDAD DE OBJETOS.

	Centro			Periferia		
Rama	pequeño (1546)	mediano (1790)	grande (634)	pequeño (1288)	mediano (2147)	grande (535)
R1	0,301	0,654	0,841	0,030	0,175	0,510
R2	0,334	0,636	0,786	0,092	0,392	0,639
R3	**0,472**	**0,806**	**0,946**	**0,296**	**0,695**	**0,892**
R4	0,451	0,747	0,885	0,191	0,480	0,739

Tabla III
EFECTOS DEL DISEÑO 2×2 EN LA PRUEBA (DIFERENCIAS DE MAP, ANILLOS POR θ).

Efecto	mAP@50 global	mAP@50 perif.	mAP@50-95 perif.
Rectificar sin *fine-tuning* (R2−R1)	+0,041	+0,147	+0,010
Rectificar con *fine-tuning* (R4−R3)	−0,138	−0,191	−0,252
Fine-tuning sin rectificar (R3−R1)	+0,282	+0,433	+0,278
Fine-tuning con rectificar (R4−R2)	+0,104	+0,096	+0,016
Interacción (R4−R3)−(R2−R1)	**−0,178**	**−0,337**	**−0,261**

en el centro a 0,510 en la periferia (-39 %) y los pequeños, de 0,301 a 0,030 (-90 %). Es decir, la deformación afecta más a los objetos que ocupan pocos píxeles. El *fine-tuning* mejora los tres tamaños. En la periferia, el mAP@50 de los objetos pequeños sube de 0,030 a 0,296.

III-B. Efectos del diseño factorial

La Tabla III compara pares de ramas que difieren en un solo factor. El *fine-tuning* es el factor más importante: sin rectificar, suma 0,433 de mAP@50 en la periferia. Rectificar mejora al detector preentrenado en 0,147, pero al detector ajustado le resta 0,191. La interacción mide cuánto cambia el efecto de rectificar según haya o no *fine-tuning*. Se calcula como $(\text{R4} - \text{R3}) - (\text{R2} - \text{R1})$ y en la periferia vale $-0{,}337$. Se esperaba que rectificar aportara menos después del *fine-tuning*, pero el resultado fue que empeora el desempeño. Esto se mantiene con otras formas de definir los anillos: la interacción es de $-0{,}339$ con el esquema de filtrado y de $-0{,}354$ con anillos por distancia al centro, cortados en un tercio y dos tercios de la media diagonal de la imagen.

III-C. Por qué rectificar resta

Las ramas rectificadas pierden sobre todo en mAP@50-95: R2 y R4 obtienen 0,155 y 0,172, contra 0,239 de R1 y 0,430 de R3. La causa es el paso de vuelta a la imagen original. Una caja recta en la proyección cilíndrica se vuelve curva en la imagen *fisheye*, y la caja que la encierra queda más grande que el objeto.

Para medir cuánto se pierde así, se simuló una detección perfecta. El polígono de cada objeto se llevó a la proyección cilíndrica, se tomó su caja y se la devolvió a la imagen *fisheye*. Aun así, la IoU mediana con la caja original fue de 0,81 en el centro y de 0,65 y 0,66 fuera de él, con anillos definidos



Figura 3. Ejemplo ilustrativo de la cámara del espejo derecho (imagen 02794_MVR, del conjunto de entrenamiento). Izquierda: referencia. Centro: R1 solo detecta las bicicletas del borde superior. Derecha: R3 detecta los autos estirados de la periferia izquierda y derecha (umbral de visualización 0,3).

Tabla IV
MAP@50 GLOBAL POR CÁMARA EN LA PRUEBA.

Rama	FV (frontal)	MVL (izq.)	MVR (der.)	RV (trasera)
R1	0,521	0,259	0,264	0,495
R2	0,498	0,354	0,363	0,474
R3	**0,701**	**0,603**	**0,653**	**0,671**
R4	0,625	0,453	0,468	0,547

por distancia al centro. Fuera del centro, entre el 6,9 % y el 11,6 % de los objetos no llega a IoU 0,5. Además, el 3,4 % de los objetos queda fuera de la banda cilíndrica y no se puede detectar.

El problema no está en el entrenamiento de R4. Sobre la proyección cilíndrica, R4 llega a 0,446 de mAP@50-95 en validación, casi lo mismo que R3 sobre la imagen *fisheye* (0,431). Lo que pierde aparece al devolver las cajas, y ningún entrenamiento lo compensa. Con el detector preentrenado, en cambio, rectificar ayuda más de lo que cuesta, al menos con IoU 0,5.

III-D. Cámaras, clases y ejemplo cualitativo

Las cámaras laterales son las más afectadas (Tabla IV), porque miran hacia abajo y ven casi toda la calle en la periferia. R3 sube su mAP@50 de 0,26 a 0,60 en MVL y a 0,65 en MVR, más que lo que logra R1 con la cámara frontal. Por clase, en la periferia, `vehicle` pasa de 0,185 a 0,816, `person` de 0,207 a 0,507 y `two_wheeler` de 0,117 a 0,486. La Fig. 3 muestra un ejemplo con una imagen del conjunto de entrenamiento. El detector de COCO no encuentra los autos estirados en los bordes de una cámara lateral, y el modelo ajustado sí.

III-E. Latencia

Las cuatro ramas se midieron en la misma máquina y en la misma sesión (Tabla I). El detector tarda entre 23 y 25 ms por imagen en todas, porque es el mismo modelo. Rectificar agrega unos 3 ms, un 12 % más, para proyectar la imagen y devolver las cajas. El *fine-tuning* no agrega tiempo, porque solo cambia los pesos. En otra sesión el mismo modelo tardó 17 ms, pero las diferencias entre ramas se mantuvieron.

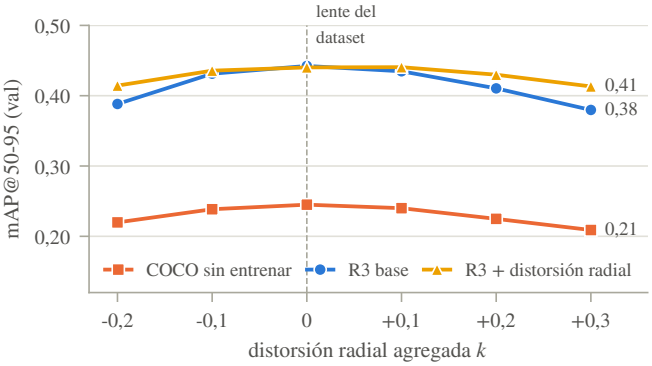


Figura 4. Robustez a una distorsión radial agregada en validación (300 imágenes). En $k = 0$ las tres curvas corresponden a la lente del conjunto de datos. Hacia la derecha la lente es más *fisheye* y hacia la izquierda, menos.

III-F. Entrenamiento y aumento de datos con distorsión radial

En R3, el mAP@50-95 de validación sube 0,123 entre las épocas 1 y 10, 0,062 entre la 10 y la 20, y 0,026 entre la 20 y la 30. Cada tramo aporta menos que el anterior. Tampoco hay sobreajuste, porque validación y prueba dan 0,652 y 0,659 de mAP@50. Entrenar más épocas sumaría centésimas, poco frente a los 0,43 que ya se ganaron en la periferia.

Con las lentes de WoodScape, el aumento de datos con distorsión radial no cambia el resultado. En la prueba da 0,656 de mAP@50 y 0,426 de mAP@50-95, contra 0,659 y 0,430 del modelo base. Su aporte aparece cuando cambia la lente (Fig. 4). Se deformaron 300 imágenes de validación con k entre $-0{,}2$ y $+0{,}3$. El modelo base pierde hasta 0,062 de mAP@50-95 y el entrenado con el aumento, 0,027, menos de la mitad.

IV. Conclusiones

Se compararon cuatro formas de usar YOLO11m sobre las cámaras *fisheye* de WoodScape, con los mismos datos y la misma métrica. De los resultados se extraen las siguientes conclusiones:

1. El problema es la deformación de la lente y no el detector elegido. YOLO11m pierde el 69 % de su mAP@50

del centro a la periferia, en objetos de todos los tamaños. Un detector más nuevo, YOLO26m, rinde casi igual en la periferia.

2. La mejor estrategia es hacer *fine-tuning* sobre la imagen *fisheye*. Sube la periferia de 0,170 a 0,603 y el mAP@50 global de 0,378 a 0,659, mejora las cuatro cámaras y no agrega latencia, porque es el mismo modelo con otros pesos.
3. Rectificar la imagen solo sirve si no se hace *fine-tuning*. Mejora al detector preentrenado en la periferia, pero empeora al modelo ajustado. Se esperaba que aportara poco después del *fine-tuning*, no que restara.
4. Rectificar resta porque las cajas vuelven deformadas a la imagen original. Aun con una detección perfecta, la caja devuelta coincide poco con el objeto fuera del centro, y algunos objetos quedan fuera de la proyección. Cualquier método que rectifique, detecte y devuelva cajas tiene este límite.
5. Deformar las imágenes al entrenar no mejora el resultado con las lentes de WoodScape. Sí hace al modelo más robusto: con otras lentes pierde la mitad que sin ese aumento de datos. Esto sirve si los vehículos usan cámaras distintas.
6. Hay que evaluar por anillos de ángulo y sobre la imagen original. El mAP global esconde lo que pasa en la periferia, y medir cada rama en su propia imagen habría escondido el costo de rectificar. Las conclusiones no cambian con otras formas de definir los anillos.

Trabajo futuro. El estudio se puede extender en tres direcciones. La primera es repetir el diseño con otros detectores, de dos etapas o basados en *transformers*. La segunda es rectificar de otra forma, con varias vistas de perspectiva o una proyección esférica. La tercera es usar cajas rotadas, elipses o polígonos, que siguen la forma deformada del objeto [4]. Así, las ramas rectificadas podrían perder menos al volver a la imagen original.

Referencias

[1] S. Yogamani, C. Hughes, J. Horgan, G. Sistu, P. Varley, D. O'Dea, M. Uřičář, S. Milz, M. Simon, K. Amende, C. Witt, H. Rashed, S. Chennupati, S. Nayak, S. Mansoor, X. Perrotton y P. Pérez, "WoodScape: A multi-task, multi-camera fisheye dataset for autonomous driving," en *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, 2019, pp. 9307–9317, doi: 10.1109/ICCV.2019.00940.

[2] V. R. Kumar, C. Eising, C. Witt y S. K. Yogamani, "Surround-view fisheye camera perception for automated driving: Overview, survey & challenges," *IEEE Trans. Intell. Transp. Syst.*, vol. 24, no. 4, pp. 3638–3659, 2023, doi: 10.1109/TITS.2023.3235057.

[3] V. R. Kumar, S. K. Yogamani, H. Rashed, G. Sistu, C. Witt, I. Leang, S. Milz y P. Mäder, "OmniDet: Surround view cameras based multi-task visual perception network for autonomous driving," *IEEE Robot. Autom. Lett.*, vol. 6, no. 2, pp. 2830–2837, 2021, doi: 10.1109/LRA.2021.3062324.

[4] H. Rashed, E. Mohamed, G. Sistu, V. R. Kumar, C. Eising, A. El-Sallab y S. Yogamani, "Generalized object detection on fisheye cameras for autonomous driving: Dataset, representations and baseline," en *Proc. IEEE/CVF Winter Conf. Appl. Comput. Vis. (WACV)*, 2021, pp. 2271–2279, doi: 10.1109/WACV48630.2021.00232.

[5] E. Plaut, E. Ben-Yaacov y B. El Shlomo, "3D object detection from a single fisheye image without a single fisheye training image," en *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. Workshops (CVPRW)*, 2021, pp. 3654–3662, doi: 10.1109/CVPRW53098.2021.00405.

[6] M. Gochoo, M.-E. Otgonbold, E. Ganbold, J.-W. Hsieh, M.-C. Chang, P.-Y. Chen, B. Dorj, H. Al Jassmi, G. Batnasan, F. Alnajjar, M. Abduljabbar y F.-P. Lin, "FishEye8K: A benchmark and dataset for fisheye camera object detection," en *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. Workshops (CVPRW)*, 2023, pp. 5305–5313, doi: 10.1109/CVPRW59228.2023.00559.

[7] J. Kannala y S. S. Brandt, "A generic camera model and calibration method for conventional, wide-angle, and fish-eye lenses," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 28, no. 8, pp. 1335–1340, 2006, doi: 10.1109/TPAMI.2006.153.

[8] T.-Y. Lin, M. Maire, S. Belongie, J. Hays, P. Perona, D. Ramanan, P. Dollár y C. L. Zitnick, "Microsoft COCO: Common objects in context," en *Proc. Eur. Conf. Comput. Vis. (ECCV)*, 2014, pp. 740–755, doi: 10.1007/978-3-319-10602-1_48.

[9] G. Jocher y J. Qiu, "Ultralytics YOLO11," versión 11.0.0, 2024. [En línea]. Disponible: https://github.com/ultralytics/ultralytics

[10] D. C. Brown, "Decentering distortion of lenses," *Photogramm. Eng.*, vol. 32, no. 3, pp. 444–462, 1966.

[11] G. Jocher, J. Qiu, M. Liu, S. Lyu, F. C. Akyon y M. E. Kalfaoglu, "Ultralytics YOLO26: Unified real-time end-to-end vision models," arXiv:2606.03748, 2026, doi: 10.48550/arXiv.2606.03748.